

CSE312: Database Management Systems Lab

Lab 03: PostgreSQL Data Manipulation Language (DML)

1. Objective

The objectives of this experiment are:

- To understand the purpose of Data Manipulation Language (DML).
- To insert single and multiple records into PostgreSQL tables.
- To retrieve and filter records for verifying DML operations.
- To update existing records using suitable conditions.
- To delete selected records without affecting unrelated data.
- To use PostgreSQL's `RETURNING` clause to inspect affected rows.
- To observe how constraints and foreign keys maintain data integrity during DML operations.

2. Introduction to DML

DML: Data Manipulation Language is the subset of SQL used to add, modify, and remove records stored in database tables.

The main PostgreSQL DML commands are:

- **INSERT:** adds one or more new rows to a table.
- **UPDATE:** changes values in existing rows.
- **DELETE:** removes selected rows from a table.

SELECT is commonly classified as Data Query Language (DQL). It is included in this lab because it is essential for inspecting data before and after a DML operation.

2.1 General Safety Rules

- Specify column names explicitly in an `INSERT` statement.
- Check a `WHERE` condition with `SELECT` before using it in `UPDATE` or `DELETE`.
- Remember that omitting `WHERE` affects every row in the table.
- Insert parent records before child records when foreign keys are present.

2.2 Commands, Clauses, and Keywords

An SQL statement is built from several parts. A **command** states the main operation, while a **clause** adds information about the target, data, condition, sorting, or output. Keywords are reserved SQL words used to form these parts.

Table 1: SQL commands, clauses, and keywords used in this experiment

Part	Function	Example
<code>INSERT INTO</code>	Starts an insertion and identifies the table that will receive new rows.	<code>INSERT INTO student</code>
Column list	Identifies the columns that will receive the supplied values.	<code>(first_name, email)</code>
<code>VALUES</code>	Introduces one or more rows of literal values to insert.	<code>VALUES ('Amina', 'a@example.com')</code>
<code>DEFAULT</code>	Uses the default value defined for a column.	<code>VALUES ('Amina', DEFAULT)</code>
<code>NULL</code>	Represents an unknown, missing, or inapplicable value.	<code>VALUES ('Amina', NULL)</code>
<code>ON CONFLICT</code>	Defines how to handle a unique-value conflict during insertion.	<code>ON CONFLICT (email)</code>
<code>DO NOTHING</code>	Skips a conflicting row instead of producing an error.	<code>ON CONFLICT (email) DO NOTHING</code>
<code>SELECT</code>	Specifies the columns or expressions to retrieve.	<code>SELECT student_id, first_name</code>
<code>FROM</code>	Identifies the table from which rows are retrieved or deleted.	<code>FROM student</code>

Part	Function	Example
WHERE	Filters rows using a true-or-false condition.	WHERE dept_id = 1
ORDER BY	Sorts query output by one or more columns.	ORDER BY first_name
ASC	Requests ascending sorting; this is the default.	ORDER BY first_name ASC
DESC	Requests descending sorting.	ORDER BY admission_year DESC
UPDATE	Starts a modification and identifies the table to change.	UPDATE student
SET	Specifies the new value of one or more columns.	SET is_active = FALSE
DELETE FROM	Starts deletion and identifies the table from which rows will be removed.	DELETE FROM enrollment
RETURNING	Displays values from rows affected by INSERT, UPDATE, or DELETE.	RETURNING student_id

2.3 Conditions and Operators

The WHERE clause uses operators to decide whether each row should be selected, updated, or deleted.

Table 2: Common operators used in WHERE conditions

Operator	Function	Example
=	Equal to	dept_id = 1
<> or !=	Not equal to	grade <> 'F'
>	Greater than	credit > 0
>=	Greater than or equal to	admission_year >= 2022
<	Less than	admission_year < 2022
<=	Less than or equal to	credit <= 3.0
AND	Requires both conditions to be true	dept_id = 1 AND is_active = TRUE
OR	Requires at least one condition to be true	grade = 'A' OR grade = 'A+'
NOT	Reverses a condition	NOT is_active

Operator	Function	Example
IS NULL	Tests whether a value is null	<code>grade IS NULL</code>
IS NOT NULL	Tests whether a value is not null	<code>email IS NOT NULL</code>

2.4 How to Read an SQL Statement

Consider the following statement:

```
SELECT student_id, first_name
FROM student
WHERE dept_id = 1
ORDER BY first_name;
```

1. `SELECT student_id, first_name` states which columns should appear in the result.
2. `FROM student` states where PostgreSQL should read the rows.
3. `WHERE dept_id = 1` keeps only students belonging to department 1.
4. `ORDER BY first_name` sorts the remaining rows by first name.

Although `SELECT` is written first, PostgreSQL conceptually identifies the source in `FROM`, applies the `WHERE` filter, forms the selected output, and then performs the `ORDER BY` sorting.

3. Preparing the Database

This experiment uses the university database structure introduced in Lab 02. Connect to the database first.

```
psql -U postgres
\c university_lab
```

If the Lab 02 tables are not available, create them using the following statements.

```
CREATE TABLE IF NOT EXISTS department (
    dept_id SERIAL PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL UNIQUE,
    location VARCHAR(100)
);
```

```

CREATE TABLE IF NOT EXISTS student (
    student_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    date_of_birth DATE,
    admission_year INTEGER CHECK (admission_year >= 2000),
    is_active BOOLEAN DEFAULT TRUE,
    dept_id INTEGER REFERENCES department(dept_id)
);

```

```

CREATE TABLE IF NOT EXISTS course (
    course_id SERIAL PRIMARY KEY,
    course_code VARCHAR(20) NOT NULL UNIQUE,
    course_title VARCHAR(150) NOT NULL,
    credit NUMERIC(3, 1) CHECK (credit > 0),
    dept_id INTEGER NOT NULL REFERENCES department(dept_id)
);

```

```

CREATE TABLE IF NOT EXISTS enrollment (
    student_id INTEGER REFERENCES student(student_id),
    course_id INTEGER REFERENCES course(course_id),
    enrollment_date DATE DEFAULT CURRENT_DATE,
    grade VARCHAR(2),
    PRIMARY KEY (student_id, course_id),
    CONSTRAINT grade_check
        CHECK (grade IN ('A+', 'A', 'B+', 'B', 'C+', 'C', 'D', 'F'))
);

```

For repeatable practice, remove existing rows and reset the generated identifiers.

```

TRUNCATE TABLE enrollment, student, course, department
RESTART IDENTITY CASCADE;

```

4. Inserting Data with INSERT

4.1 Inserting a Single Row

INSERT: The INSERT command creates one or more new rows in a table.

General syntax:

```
INSERT INTO table_name (column_1, column_2, ...)
VALUES (value_1, value_2, ...);
```

Clause breakdown:

- **INSERT INTO:** begins the insertion operation.
- **table_name:** identifies the table that will receive the new row.
- **(column_1, column_2, ...):** lists the target columns in the order in which values will be supplied.
- **VALUES:** introduces the actual data for the new row.
- **(value_1, value_2, ...):** supplies one value for each listed column.
- The semicolon terminates the SQL statement.

Text and date values are enclosed in single quotation marks. Numeric values are normally written without quotation marks. The number and order of values must match the column list.

```
INSERT INTO department (dept_name, location)
VALUES ('Computer Science and Engineering', 'Academic Building 1');
```

In this example, `department` is the target table. The first value is stored in `dept_name`, and the second value is stored in `location`. The `dept_id` column is omitted because PostgreSQL generates its `SERIAL` value.

4.2 Inserting Multiple Rows

Multiple records can be added with one INSERT statement. Each parenthesized group after **VALUES** represents one row, and commas separate the row groups.

```
INSERT INTO department (dept_name, location)
VALUES
    ('Electrical and Electronic Engineering', 'Academic Building 2'),
    ('Business Administration', 'Academic Building 3');
```

This form is shorter and usually more efficient than executing a separate `INSERT` statement for every row. Every row group must contain values compatible with the same column list.

4.3 Inserting Rows with Optional and Default Values

Columns that have a default value may be omitted. Here, `is_active` receives its default value of `TRUE`.

```
INSERT INTO student
    (first_name, last_name, email, date_of_birth,
     admission_year, dept_id)
VALUES
    ('Amina', 'Rahman', 'amina.rahman@example.com',
     '2003-02-12', 2022, 1);
```

Use `DEFAULT` explicitly when it makes the intention clearer.

```
INSERT INTO student
    (first_name, last_name, email, date_of_birth,
     admission_year, is_active, dept_id)
VALUES
    ('Nafis', 'Ahmed', 'nafis.ahmed@example.com',
     '2002-09-25', 2021, DEFAULT, 1);
```

Keyword breakdown:

- `DEFAULT`: tells PostgreSQL to use the default defined for that column.
- An omitted column also receives its default value when one exists.
- If an omitted column has no default, PostgreSQL normally stores `NULL`, provided the column allows null values.
- `NULL`: represents a missing, unknown, or not-yet-applicable value; it is not an empty string or zero.

4.4 Inserting Additional Sample Data

```
INSERT INTO student
    (first_name, last_name, email, date_of_birth,
     admission_year, dept_id)
VALUES
```

```

('Sadia', 'Islam', 'sadia.islam@example.com',
 '2003-07-08', 2022, 2),
('Tanvir', 'Hasan', 'tanvir.hasan@example.com',
 '2004-01-19', 2023, 1),
('Maliha', 'Chowdhury', 'maliha.c@example.com',
 '2002-11-03', 2021, 3);

INSERT INTO course (course_code, course_title, credit, dept_id)
VALUES
('CSE312', 'Database Management Systems Lab', 1.5, 1),
('CSE311', 'Database Management Systems', 3.0, 1),
('EEE201', 'Electrical Circuits', 3.0, 2),
('BUS101', 'Introduction to Business', 3.0, 3);

INSERT INTO enrollment
(student_id, course_id, enrollment_date, grade)
VALUES
(1, 1, '2026-01-15', 'A'),
(1, 2, '2026-01-15', 'B+'),
(2, 1, '2026-01-16', 'A+'),
(3, 3, '2026-01-17', 'B'),
(4, 1, DEFAULT, NULL),
(5, 4, DEFAULT, 'A');

```

4.5 Returning Inserted Values

PostgreSQL can immediately return values generated or stored by an INSERT.

General syntax:

```

INSERT INTO table_name (columns)
VALUES (values)
RETURNING column_1, column_2, ...;

INSERT INTO student
(first_name, last_name, email, admission_year, dept_id)
VALUES
('Rafi', 'Karim', 'rafi.karim@example.com', 2024, 1)
RETURNING student_id, first_name, last_name, is_active;

```

Clause breakdown:

- **RETURNING**: asks PostgreSQL to output data from the affected row immediately.
- The listed columns determine which values are displayed.
- Generated values such as `student_id` can be obtained without a separate **SELECT**.
- **RETURNING *** may be used to return every column of the inserted row.

4.6 Handling a Duplicate Value

The email column is unique. PostgreSQL's **ON CONFLICT** clause can safely ignore a duplicate.

General syntax:

```
INSERT INTO table_name (columns)
VALUES (values)
ON CONFLICT (conflict_column) DO NOTHING;

INSERT INTO student
    (first_name, last_name, email, admission_year, dept_id)
VALUES
    ('Amina', 'Rahman', 'amina.rahman@example.com', 2022, 1)
ON CONFLICT (email) DO NOTHING;
```

Clause breakdown:

- **ON CONFLICT**: defines what PostgreSQL should do if the insertion violates a suitable unique or exclusion constraint.
- **(email)**: identifies the unique column used to detect the conflict.
- **DO NOTHING**: skips the conflicting row instead of raising a duplicate-value error.
- Non-conflicting rows in the same multi-row insertion can still be inserted.

5. Viewing Data with SELECT

5.1 Displaying All Rows

SELECT: The **SELECT** command retrieves data from one or more tables without changing the stored records.

General syntax:

```
SELECT column_1, column_2, ...  
FROM table_name  
WHERE condition  
ORDER BY column_name;
```

Clause breakdown:

- **SELECT:** specifies the columns or expressions to display.
- *****: selects all columns from the table.
- **FROM:** identifies the table from which rows are read.
- **WHERE:** keeps only rows that satisfy a condition; it is optional.
- **ORDER BY:** sorts the result; it is also optional.

```
SELECT * FROM department;  
SELECT * FROM student;  
SELECT * FROM course;  
SELECT * FROM enrollment;
```

5.2 Selecting Particular Columns

```
SELECT student_id, first_name, last_name, admission_year  
FROM student;
```

A comma-separated column list limits the output to the required attributes. This improves readability and avoids retrieving unnecessary data.

5.3 Filtering and Sorting Rows

```
SELECT student_id, first_name, last_name, admission_year  
FROM student  
WHERE dept_id = 1 AND admission_year >= 2022  
ORDER BY admission_year, first_name;
```

Clause and operator breakdown:

- **WHERE dept_id = 1:** selects rows whose department identifier equals 1.
- **=:** tests equality.
- **>=:** tests whether the left value is greater than or equal to the right value.
- **AND:** requires both connected conditions to be true.

- `ORDER BY admission_year, first_name`: sorts first by admission year and then by first name when years are equal.
- Ascending order is the default. `DESC` may be added after a column for descending order.

Other common comparison operators are `<>` or `!=` for “not equal,” `<` for “less than,” and `>` for “greater than.” The logical operator `OR` requires at least one connected condition to be true, while `NOT` reverses a condition.

5.4 Checking Null Values

The `IS NULL` operator finds enrollments for which a grade has not yet been assigned.

```
SELECT student_id, course_id, enrollment_date
FROM enrollment
WHERE grade IS NULL;
```

`NULL` cannot be tested reliably with `= NULL`. Use `IS NULL` to find null values and `IS NOT NULL` to find rows containing a known value.

6. Modifying Data with UPDATE

6.1 Updating One Row

UPDATE: The `UPDATE` command changes values in rows that already exist in a table.

General syntax:

```
UPDATE table_name
SET column_1 = new_value,
    column_2 = new_value
WHERE condition
RETURNING columns;
```

Clause breakdown:

- `UPDATE`: begins the modification operation.
- `table_name`: identifies the table whose rows may be changed.
- `SET`: introduces one or more column assignments.

- `column = new_value`: assigns a replacement value to a column.
- Commas separate multiple assignments in the same `SET` clause.
- `WHERE`: selects which rows will be updated.
- `RETURNING`: optionally displays values from the changed rows.

```
UPDATE student
SET email = 'nafis.a@example.com'
WHERE student_id = 2;
```

The command searches `student` for the row whose `student_id` is 2 and changes only its `email`. If no row satisfies the condition, PostgreSQL reports `UPDATE 0`, and no data is changed.

6.2 Updating Multiple Columns

```
UPDATE department
SET dept_name = 'Business Studies',
    location = 'Administration Building'
WHERE dept_id = 3;
```

Both assignments apply to every row selected by the same `WHERE` condition. The new values must satisfy the columns' data types and constraints.

6.3 Updating Multiple Rows

The following statement deactivates students admitted before 2022.

```
UPDATE student
SET is_active = FALSE
WHERE admission_year < 2022;
```

A `WHERE` condition does not have to identify only one row. In this example, every student admitted before 2022 is updated.

6.4 Updating a Value Using Its Current Value

```
UPDATE course
SET credit = credit + 0.5
WHERE course_code = 'CSE312';
```

The expression on the right side is evaluated using the row's current value. Therefore, `credit = credit + 0.5` increases the existing credit rather than replacing it with a fixed number.

6.5 Updating and Returning Changed Rows

```
UPDATE enrollment
SET grade = 'B+'
WHERE student_id = 4 AND course_id = 1
RETURNING student_id, course_id, grade;
```

The two conditions joined by **AND** identify one enrollment through its composite primary key. After the update, **RETURNING** displays the identifiers and the new grade.

6.6 The Effect of Omitting WHERE

The following statement would change every student record. It is shown only as a warning and should not be executed during this experiment.

```
-- Dangerous: affects every row in student.
UPDATE student SET is_active = FALSE;
```

Without **WHERE**, the target set is the entire table. Before a broad update, run a **SELECT** with the intended condition and confirm that it returns exactly the rows that should be changed.

7. Removing Data with DELETE

7.1 Deleting One Selected Row

DELETE: The **DELETE** command removes existing rows from a table while leaving the table structure unchanged.

General syntax:

```
DELETE FROM table_name
WHERE condition
RETURNING columns;
```

Clause breakdown:

- **DELETE FROM:** begins the deletion and identifies the target table.
- **WHERE:** selects the rows to remove.
- **RETURNING:** optionally displays values from the deleted rows.

- Unlike `SELECT`, a basic `DELETE` does not place a column list between `DELETE` and `FROM`.

First inspect the target row with the same `WHERE` condition, and then delete it.

```
SELECT *  
FROM enrollment  
WHERE student_id = 5 AND course_id = 4;
```

```
DELETE FROM enrollment  
WHERE student_id = 5 AND course_id = 4;
```

The first statement only previews the row. The second statement removes the selected row.

7.2 Deleting Multiple Rows

```
DELETE FROM enrollment  
WHERE grade = 'F';
```

Every enrollment whose grade equals 'F' is selected. The number of removed rows is not fixed; it depends on the current table contents.

7.3 Deleting and Returning Removed Rows

```
DELETE FROM student  
WHERE student_id = 6  
RETURNING student_id, first_name, last_name;
```

For `DELETE`, `RETURNING` shows the old values from rows that have just been removed. This is useful for verification and application logging.

7.4 Foreign Key Restrictions

A department cannot normally be deleted while students or courses still reference it.

```
DELETE FROM department  
WHERE dept_id = 1;
```

If PostgreSQL reports a foreign key violation, remove or update the related child records first. The restriction prevents records from referring to a department that no longer exists.

7.5 DELETE Compared with TRUNCATE

Feature	DELETE	TRUNCATE
SQL category	DML	DDL in many introductory classifications
Row selection	Supports WHERE	Removes all rows
Row output	Supports RETURNING	Does not return removed rows
Identity reset	Sequence is unchanged	Can reset generated sequences
Typical use	Remove selected records	Quickly empty a table

Table 3: Comparison of DELETE and TRUNCATE

TRUNCATE TABLE is useful when every row must be removed. Its optional clauses used earlier in this manual have the following purposes:

- **RESTART IDENTITY**: resets sequences owned by the truncated tables, so generated identifiers begin again.
- **CASCADE**: also truncates tables that have foreign-key dependencies on the named tables.
- **CASCADE** must be used carefully because it can remove data from additional tables.

8. Observing Data Integrity

Constraints reject DML operations that would store invalid or inconsistent data. Execute the following statements one at a time and study the PostgreSQL error messages.

8.1 Unique Constraint Violation

```
INSERT INTO department (dept_name, location)
VALUES ('Computer Science and Engineering', 'Another Building');
```

The `dept_name` value already exists. Because the column has a **UNIQUE** constraint, PostgreSQL rejects the complete statement and does not insert a partial row.

8.2 Check Constraint Violation

```
INSERT INTO course (course_code, course_title, credit, dept_id)
VALUES ('CSE400', 'Invalid Credit Course', -1.0, 1);
```

The value `-1.0` does not satisfy `CHECK (credit > 0)`. The `VALUES` clause is syntactically correct, but the proposed row violates a business rule defined in the table.

8.3 Foreign Key Constraint Violation

```
INSERT INTO enrollment (student_id, course_id, grade)
VALUES (999, 1, 'A');
```

The foreign-key value `student_id = 999` must match an existing `student.student_id`. PostgreSQL rejects the enrollment because its parent student does not exist.

8.4 Not-Null Constraint Violation

```
INSERT INTO student (last_name, admission_year, dept_id)
VALUES ('No First Name', 2024, 1);
```

The column list omits `first_name`. Since that column has no default and is declared `NOT NULL`, PostgreSQL cannot create the row.

9. Observations

- Single and multiple records were inserted using `INSERT`.
- Default values and generated identifiers reduced the number of values that had to be supplied.
- `SELECT` statements were used to inspect and verify table records.
- `UPDATE` modified only the rows that satisfied its `WHERE` condition.
- `DELETE` removed selected records while foreign keys protected related data.
- The `RETURNING` clause displayed affected rows without requiring a separate query.
- Database constraints rejected duplicate, invalid, incomplete, and unrelated records.

10. Lab Task

Task A: Populate the Library Database

Use the `library_lab` database and the `publisher`, `book`, `member`, and `borrow` tables created in Lab 02.

1. Insert at least three publishers.
2. Insert at least six books distributed among the publishers.
3. Insert at least five library members.
4. Insert at least five borrowing records.
5. Allow at least one optional column to use its default value.
6. Use a multi-row `INSERT` statement at least once.
7. Display all records from each table.

Task B: Update Library Records

Perform the following operations using appropriate `WHERE` conditions:

1. Change the contact information of one member.
2. Update the title or price of one book.
3. Mark one borrowed book as returned.
4. Update more than one book record using a single statement.
5. Use `RETURNING` in at least one `UPDATE`.
6. Display the affected records after each operation.

Task C: Delete Practice

1. Insert a temporary member and display the generated identifier using `RETURNING`.
2. Display the temporary member using its generated identifier.
3. Delete the temporary member using an appropriate `WHERE` condition.
4. Use `RETURNING` to display the deleted member's identifier and name.
5. Verify with `SELECT` that the record no longer exists.
6. Attempt to delete a publisher that is referenced by a book and explain the result.

Task D: Online Shop DML Problem

Assume an online shop database contains the following tables: `customer`, `product`, `orders`, and `order_item`. Write and execute PostgreSQL statements to complete these requirements:

- Insert at least four customers and five products.
- Insert two orders, each containing at least two products.
- Reduce the stock quantity of the ordered products.
- Change the status of one order from `Pending` to `Shipped`.
- Delete one product that has never been included in an order.
- Use `RETURNING` to display the identifier of a newly inserted order.
- Write verification queries that display the final contents of all four tables.

11. Conclusion

This experiment introduced PostgreSQL Data Manipulation Language through practical operations on related tables. Students inserted, viewed, updated, and deleted records; used the `RETURNING` clause; and observed constraint and foreign key behavior. These techniques provide the foundation for safely maintaining data in relational database applications.